MyCodeSchool

Teorie — primii pași în Python

Python · fișa 1 · print, tipuri de date, f-string, operatori, if / elif / else

0. print

`print` scrie pe ecran ce îi dai între paranteze, apoi trece pe rândul următor. Mai multe valori le desparți prin virgulă, iar Python pune singur **un spațiu** între ele. `sep=` schimbă ce se pune între valori (implicit `" "`), iar `end=` ce se pune la sfârșit (implicit `"\n"`, adică rând nou). `\n` scris în text e tot un rând nou. Ce e între ghilimele se afișează **exact cum e**; ce e fără ghilimele se **calculează** întâi. Un f-string (`f` în fața ghilimelelor) calculează ce e între acolade și pune valoarea în text.

```
print("Biblioteca", 2026)
print("carti", "reviste", sep=" / ")
print("Se incarca", end="... ")
print("gata")
print("Linia 1\nLinia 2")
print("10 * 4")
print(10 * 4)
print(f"Imprumutate: {10 * 4} carti")
```

```
Biblioteca 2026
carti / reviste
Se incarca... gata
Linia 1
Linia 2
10 * 4
40
Imprumutate: 40 carti
```

Capcana: ghilimelele. Fără ghilimele, Python crede că `Biblioteca` e un nume și îl caută. Fără `f` în față, acoladele sunt doar text și nu se calculează nimic, fără nicio eroare.

```
print(Biblioteca)
```

```
NameError: name 'Biblioteca' is not defined
```

```
print("Total: {10 * 4}")
```

```
Total: {10 * 4}
```

1. Variabile și tipuri de date

O variabilă e un **nume lipit pe o valoare**: `zile = 7` pune numele `zile` pe valoarea `7`. Valoarea are un **tip**, iar tipul decide ce poți face cu ea. `type()` îți spune tipul.

cod

```
oras = "Cluj"
temperatura = 21.5
zile = 7
ploua = False
print(oras, type(oras))
print(temperatura, type(temperatura))
print(zile, type(zile))
print(ploua, type(ploua))
```

ieșire

```
Cluj <class 'str'>
21.5 <class 'float'>
7 <class 'int'>
False <class 'bool'>
```

tip	ce ține	exemplu
`int`	număr întreg	`7, -3, 0`
`float`	număr cu zecimale (cu punct, nu virgulă)	`21.5, 4.0`
`str`	text, între ghilimele	`"Cluj", "7"`
`bool`	adevărat / fals, cu literă mare	`True, False`

Conversii. `int()`, `float()` și `str()` nu schimbă valoarea veche, ci **fac una nouă**, de alt tip. `7` și `"7"` arată la fel pe ecran, dar pentru Python sunt lucruri diferite: `+` pe numere adună, `+` pe text lipește.

```
print("8" + "1")
print(int("8") + 1)
print(float("4.5") * 2)
print(str(3) + " kg")
```

```
81
9
9.0
3 kg
```

Capcana: `input()` întoarce mereu `str`. Tastatura trimite caractere, nu numere. Python nu ghicește ce ai vrut, deci `"15" * 2` repetă textul, fără nicio eroare:

```
pret = input("Pret: ")
print(type(pret))
print(pret * 2)
print(float(pret) * 2)
```

```
Pret: 15
<class 'str'>
1515
30.0
```

Și conversia are reguli: `int("7.5")` nu rotunjește, dă `ValueError: invalid literal for int() with base 10: '7.5'`. Pentru zecimale folosești `float()`.

2. f-string — text cu valori în el

Un f-string e un text cu `f` lipit în fața ghilimelelor. Ce scrii între acolade `{ }` Python **calculează** și pune rezultatul în text, oricare ar fi tipul lui: o variabilă sau un calcul. Restul textului rămâne exact cum l-ai scris, cu spații, paranteze sau `%`. Față de virgule, **tu** decizi unde vin spațiile.

```
produs = "Ceai"
pret = 12.5
bucati = 4
procent = 10
print("Total:", pret * bucati, "lei")
print(f"Total: {pret * bucati} lei")
print(f"{bucati} x {produs} = {pret * bucati} lei")
print(produs, "(", bucati, ")")
print(f"{produs}({bucati})")
print(f"Reducere {procent}%")
```

```
Total: 50.0 lei
Total: 50.0 lei
4 x Ceai = 50.0 lei
Ceai ( 4 )
Ceai(4)
Reducere 10%
```

Capcana: `+` între text și număr. `+` lipește doar text cu text. Cu un număr programul se oprește cu eroare; f-string-ul face singur conversia.

```
print("Pret: " + pret)
```

```
TypeError: can only concatenate str (not "float") to str
```

```
print(f"Pret: {pret}")
```

```
Pret: 12.5
```

3. Operatori aritmetici

Cu 23 de elevi împărțiți în echipe de câte 4:

```
elevi = 23
print(elevi / 4)
print(elevi // 4)
print(elevi % 4)
print(10 / 2)
print(2 ** 10)
```

```
5.75
5
3
5.0
1024
```

operator	ce face	rezultatul
`+` `-` `*`	adunare, scădere, înmulțire	`int` dacă ambele sunt `int`
`/`	împărțire exactă	**mereu** `float`, chiar și `10 / 2` → `5.0`
`//`	câtul: câte echipe **pline**	partea întreagă, fără zecimale
`%`	restul: câți elevi **rămân**	de la 0 până la împărțitor – 1
`**`	putere	`2 ** 10` → `1024`

Pe cifre, împărțitorul e `10`: `% 10` îți dă ultima cifră, `// 10` o taie. Iar `% 2` e `0` exact la numerele pare.

```
an = 2026
print(an % 10)
print(an // 10)
print(an % 2)
```

```
6
202
0
```

Capcana: ordinea operațiilor. Python face întâi `**`, apoi `* / // %`, abia apoi `+ -`. Media notelor 8 și 10, scrisă fără paranteze, nu e media:

```
print(8 + 10 / 2)
print((8 + 10) / 2)
print(2 + 3 * 2 ** 2)
```

```
13.0
9.0
14
```

4. Operatori de comparare

O comparație e o întrebare cu răspuns da/nu. Python o **calculează** ca pe orice operație, iar rezultatul e o valoare de tip `bool`, pe care o poți afișa sau ține într-o variabilă.

```
pret = 49.99
print(pret < 50)
print(pret == 50)
print(pret != 50)
rezultat = pret >= 50
print(rezultat, type(rezultat))
print("10" == 10)
```

```
True
False
True
False <class 'bool'>
False
```

==	!=	<	<=	>	>=
egal	diferit	mai mic	mai mic sau egal	mai mare	mai mare sau egal

Ultima linie: `"10"` e text, `10` e număr, deci nu sunt egale. Un `input()` neconvertit nu va fi niciodată `== 10`.

Capcana: `=` nu e `==`. `=` **pune** o valoare pe un nume (atribuire); `==` **întreabă** dacă două valori sunt egale. Într-un `if` se pune o întrebare, deci:

```
nota = 10
if nota = 10:
    print("Maxim")
```

```
SyntaxError: invalid syntax.
Maybe you meant '==' or ':='
instead of '='?
```

5. Operatori logici

`and`, `or` și `not` leagă valori `bool` și dau tot un `bool`:

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Pe scurt: `and` e `True` doar când **ambele** sunt; `or` e `True` când **cel puțin una** e. Un interval („temperatura între 18 și 25”) înseamnă două condiții legate cu `and`; Python acceptă și forma scurtă, ca la matematică.

```
t = 21
print(18 <= t and t <= 25)
print(18 <= t <= 25)
print(t < 0 or t > 35)
print(not t > 30)
```

```
True
True
False
True
```

Capcana: `or` în loc de `and` la interval. Orice număr e ori peste 18, ori sub 25, deci condiția cu `or` e adevărată și la 40 de grade:

```
t = 40
print(t >= 18 or t <= 25)
print(t >= 18 and t <= 25)
```

```
True
False
```

6. if / elif / else

Python verifică ramurile **de sus în jos** și o execută doar pe **prima** a cărei condiție e `True`; pe celelalte le sare, fără să le mai verifice. `else` nu are condiție: prinde **toate celelalte cazuri**. Blocul unei ramuri e format din rândurile indentate cu 4 spații sub ea, după `:`.

Livrarea unei comenzi: de la 300 lei e gratuită, de la 100 lei costă 10 lei, sub 100 costă 20 lei.

```
total = 350
if total >= 300:
    print("Livrare gratuita")
elif total >= 100:
    print("Livrare: 10 lei")
else:
    print("Livrare: 20 lei")
```

```
Livrare gratuita
```

Capcana 1: ordinea pragurilor. Inversează primele două ramuri: 350 trece deja de `>= 100`, deci prima ramură câștigă și `>= 300` nu mai e verificat niciodată. Fără eroare, doar rezultat greșit. La praguri pornești de la **cel mai mare**.

```
total = 350
if total >= 100:
    print("Livrare: 10 lei")
elif total >= 300:
    print("Livrare gratuita")
```

```
Livrare: 10 lei
```

Capcana 2: indentarea. Indentarea nu e decor, ea spune unde se termină blocul. Un rând scris la marginea din stânga **nu mai face parte** din `if` și rulează mereu; un `if` fără niciun rând indentat sub el e eroare.

```
total = 50
if total >= 100:
    print("Livrare: 10 lei")
print("Comanda trimisa")
```

```
Comanda trimisa
```

```
if total >= 300:
print("Livrare gratuita")
```

```
IndentationError:
expected an indented
block after 'if'
```

7. Funcția ca „container” pentru o problemă

`def raport_vreme():` nu rulează nimic: îi spune doar lui Python „ține minte rândurile indentate de sub mine, sub numele ăsta”. Ele rulează abia când **chemi** funcția, cu paranteze: `raport_vreme()`; fără paranteze e doar numele ei, nu se întâmplă nimic și nu primești nicio eroare. Problemele stau în `tipuri_operatori_if.py`; programul pornește din `main.py`, care le aduce cu `from tipuri_operatori_if import ...`. Când rulezi un fișier, Python îi dă numele intern `"__main__"`, iar unui fișier adus cu `import` îi dă numele lui (`"tipuri_operatori_if"`). De aceea `if __name__ == "__main__":` pornește programul doar când rulezi chiar `main.py`.

tipuri_operatori_if.py

```
def raport_vreme():
    oras = "Iasi"
    temperatura = 12.5
    print(f"{oras}: {temperatura} grade")
```

ieșire (rulezi main.py)

```
Iasi: 12.5 grade
```

main.py

```
from tipuri_operatori_if import raport_vreme


def main():
    raport_vreme()


if __name__ == "__main__":
    main()
```

Greșelile pe care le vei face — verifică-le întâi:

1. Ai citit cu `input()` și n-ai convertit → `"15" * 2` e `"1515"`, iar `"10" == 10` e `False`.
2. Ai vrut zecimale și ai scris `//`, sau ai vrut un întreg și ai scris `/` (care dă mereu `float`).
3. `if nota = 10:` → `SyntaxError`. În condiție se compară cu `==`.
4. Praguri în ordine greșită → prima ramură adevărată câștigă. Pornești de la cel mai mare.
5. Un rând fără cele 4 spații a ieșit din bloc; un apel fără `()` nu cheamă nimic.
6. Ai scris `{...}` fără `f` în fața ghilimelelor → acoladele apar ca text, nu se calculează.
7. Ai lipit text cu număr prin `+` → `TypeError`. Scrie un f-string sau pune virgulă.